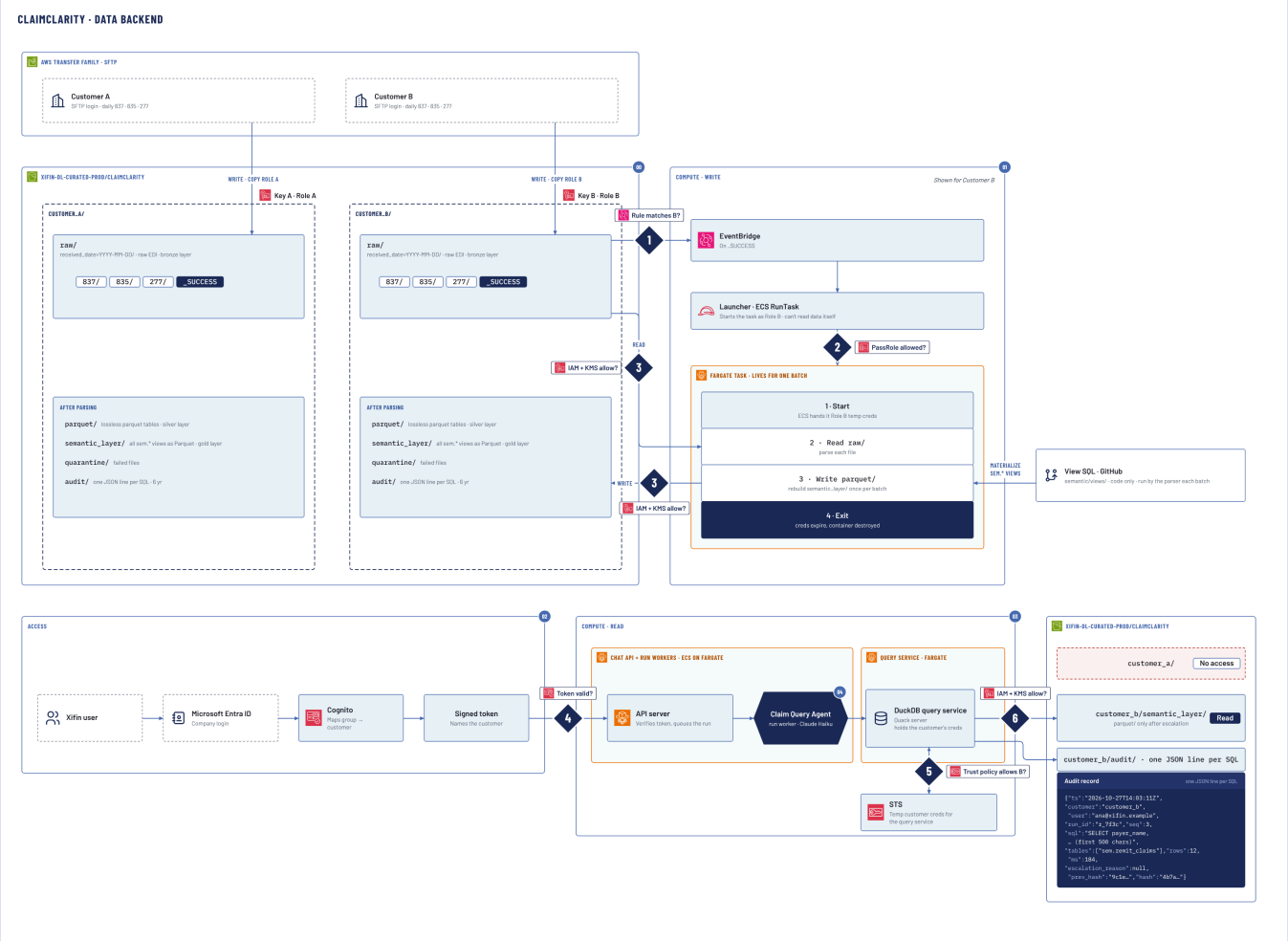
ClaimClarity · Data backend architecture

How customer claim data moves through the ClaimClarity backend, and how each customer's data is kept separate.

Overview



Summary: The whole data backend on one board, shown for Customer B. Customers send daily files over SFTP. A Fargate parser turns them into tables in the customer's folder. A Xifin user signs in, asks a question, and the Claim Query Agent answers it through the DuckDB query service. Six numbered gates (1–6) keep every step inside one customer's data.

Key components:

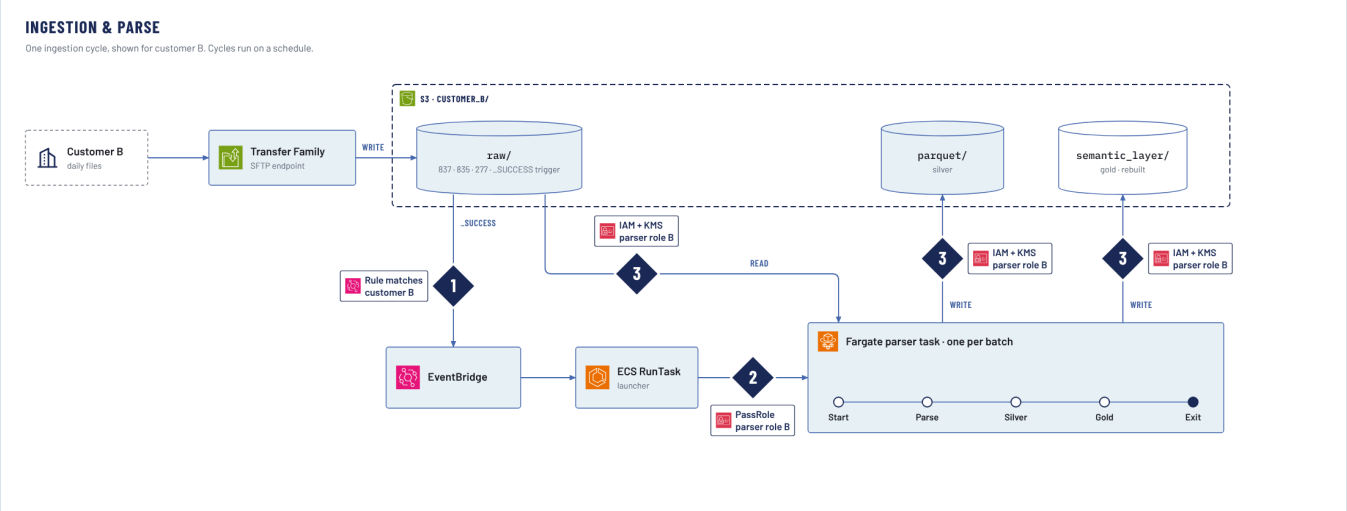
- **AWS Transfer Family (SFTP)**: each customer logs in and drops daily 837, 835 and 277 files.
- **Data bucket** `xifin-dl-curated-prod/claimclarity` : one folder per customer (`customer_a/` , `customer_b/`), each with its own KMS key and roles.
- **EventBridge**: fires when a `_SUCCESS` file lands in `raw/` .
- **Launcher (ECS RunTask)**: starts the parser task as Role B. It can't read data itself.

- **Fargate parser task**: lives for one batch. Reads `raw/`, writes `parquet/`, rebuilds `semantic_layer/` once per batch, then exits.
- **View SQL (GitHub, `semantic/views/`)**: code only. The parser runs it each batch to build the `sem.*` views.
- **Microsoft Entra ID**: company login for the Xifin user.
- **Cognito**: maps the user's group to a customer and issues a signed token naming that customer.
- **API server (Chat API + run workers, ECS on Fargate)**: verifies the token and queues the run.
- **Claim Query Agent**: runs in the run worker on Claude Haiku. Has no access to the data itself.
- **DuckDB query service (Quack server, Fargate)**: the only part that holds the customer's credentials and reads data.
- **STS**: gives the query service temporary credentials for one customer.
- `audit/`: one JSON line per SQL statement.

Notes / TODO:

- The board has drill-down badges for 00–04 only. There is no link to 05 (Access matrix).
- The board says the parser "rebuilds" `semantic_layer/`. Board 05 says it writes "a new version" and the reader sees "current version only". Agree on one wording.
- Lane labels are Compute · Write, Access, Compute · Read. Add a short legend if readers find the gate badges and arrows unclear.

01 · Ingestion & parse



Summary: One ingestion cycle for Customer B. Cycles run on a schedule. Files arrive over SFTP into `raw/`. The `_SUCCESS` file triggers one Fargate parser task, which writes the silver and gold layers and then exits.

Key components:

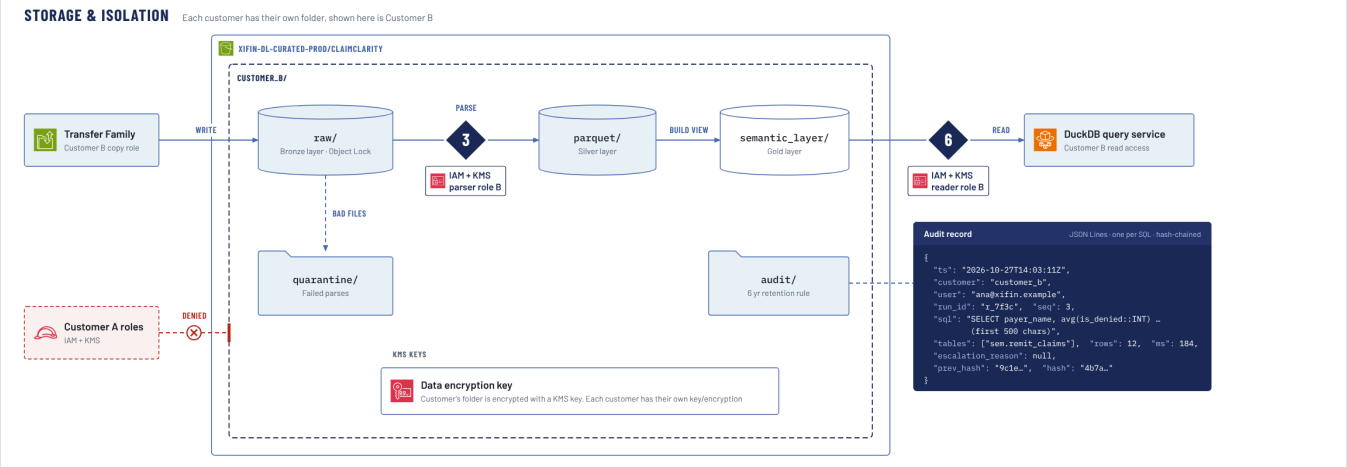
- **Customer B**: sends daily 837, 835 and 277 files.
- **Transfer Family SFTP endpoint**: writes the files into `raw/` with the Customer B copy role.
- `raw/`: holds the 837, 835 and 277 files plus the `_SUCCESS` trigger.
- **EventBridge**: sees `_SUCCESS` and calls the launcher (gate 1: rule matches customer B).
- **ECS RunTask launcher**: starts the parser task with parser role B (gate 2: PassRole allowed).
- **Fargate parser task (one per batch)**: steps are Start, Parse, Silver, Gold, Exit. Every read and write passes gate 3 (IAM + KMS for parser role B).

- `parquet/` **(silver)**: parsed tables.
- `semantic_layer/` **(gold)**: rebuilt from the view SQL each batch.

Notes / TODO:

- The board doesn't name its schedule. Confirm how often cycles run and whether `_SUCCESS` or a schedule starts them (or both).
- Failed files go to `quarantine/` (from board 00). Confirm retries and alerting.
- What happens with a missing `_SUCCESS` or a partial batch?
- Board title has no "01 ·" prefix, unlike 03–05.

00 · Storage & isolation



Summary: Each customer has their own folder in the one data bucket. This board shows Customer B's folder, the roles that use it and its KMS key. Customer A's roles are denied by IAM + KMS.

Key components:

- `raw/` : bronze layer, under Object Lock. Written by the Customer B copy role (Transfer Family).
- `parquet/` : silver layer. Written by parser role B.
- `semantic_layer/` : gold layer. Built from views by parser role B. Read by the DuckDB query service.
- `quarantine/` : failed parses.
- `audit/` : one JSON Lines record per SQL, hash-chained, with a 6-year retention rule.
- **Data encryption key (KMS)**: each customer's folder has its own key.
- **Gate 3**: IAM + KMS for parser role B.
- **Gate 6**: IAM + KMS for reader role B (DuckDB query service, Customer B read access).
- **Customer A roles**: denied on `customer_b/` by IAM + KMS.

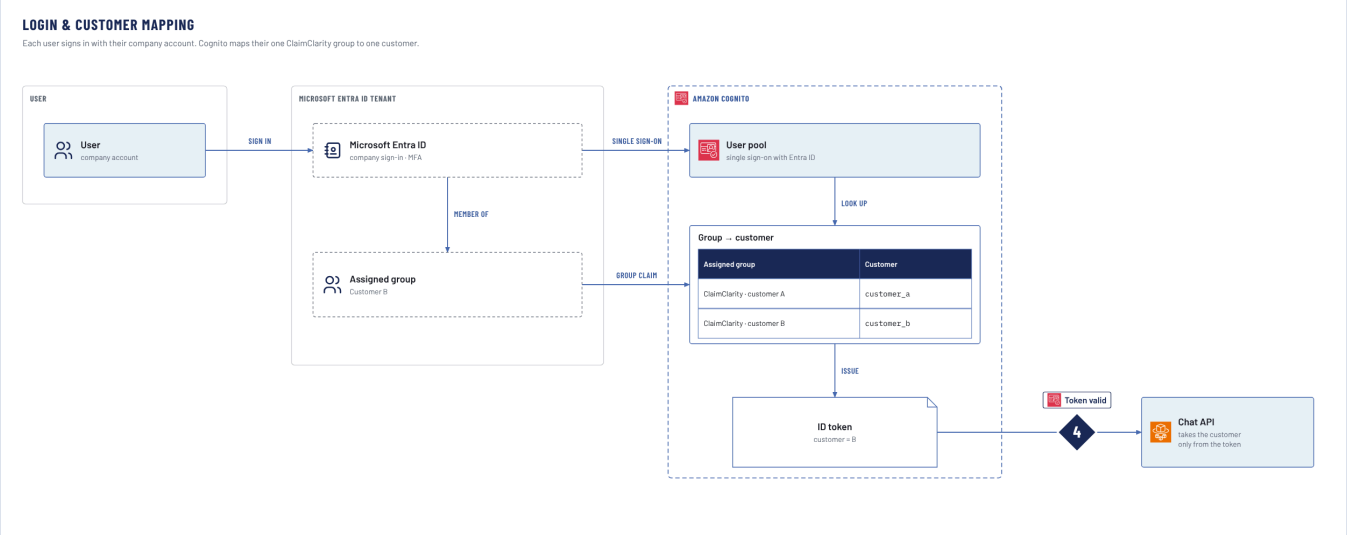
Example audit record (one line per SQL):

```
{"ts": "2026-10-27T14:03:11Z", "customer": "customer_b", "user": "ana@xifin.example",
 "run_id": "r_7f3c", "seq": 3, "sql": "SELECT payer_name, … (first 500 chars)",
 "tables": ["sem.remit_claims"], "rows": 12, "ms": 184,
 "escalation_reason": null, "prev_hash": "9c1e…", "hash": "4b7a…"}
```

Notes / TODO:

- The board doesn't show who writes `audit/`. Board 05 says the audit role (in the DuckDB query service) appends.
- Verify against the real bucket and key policies: Object Lock mode on `raw/`, the 6-year rule, key rotation.
- How is the hash chain checked, and who can read audit logs?
- Board title has no "00 ·" prefix.

02 · Login & customer mapping



Summary: Each user signs in with their company account at Microsoft Entra ID. Cognito maps their one ClaimClarity group to one customer. It issues an ID token that names that customer, and the Chat API trusts only that.

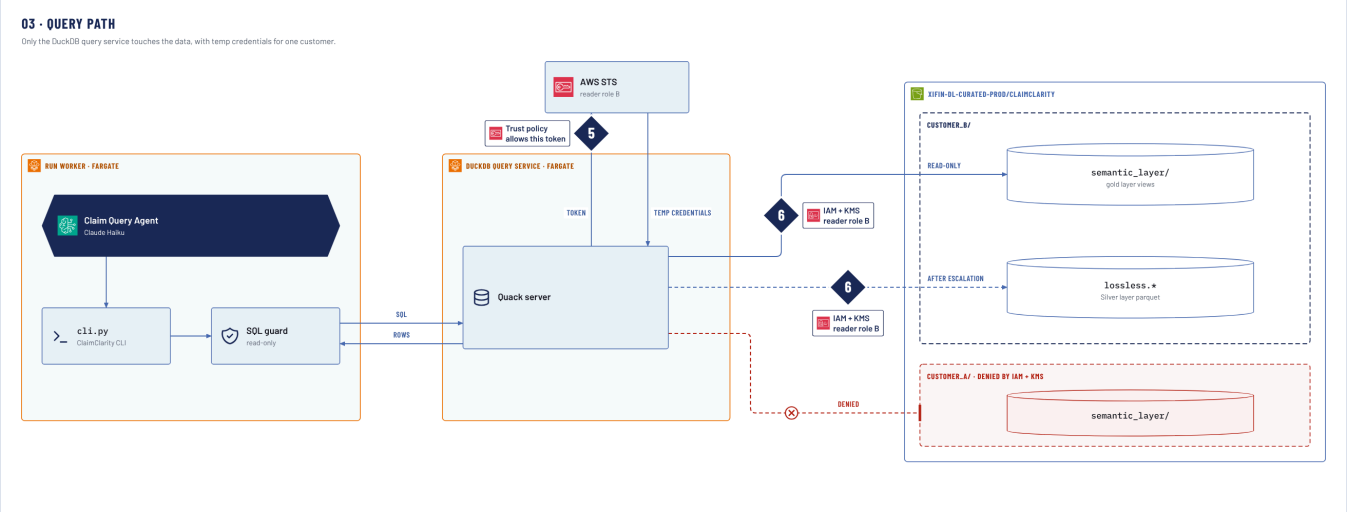
Key components:

- **User**: signs in with a company account.
- **Microsoft Entra ID**: company sign-in with MFA. The user is a member of one assigned group (e.g. Customer B).
- **Cognito user pool**: single sign-on with Entra ID. Reads the group claim and looks it up in the group → customer table.
- **Group → customer table**: "ClaimClarity · customer A" → `customer_a`, "ClaimClarity · customer B" → `customer_b`.
- **ID token**: carries `customer = B`.
- **Chat API**: takes the customer only from the token (gate 4: token valid).

Notes / TODO:

- Who owns the group → customer table, and how is a new customer added?
- What happens if a user has zero groups or more than one?
- Board title has no "02 ·" prefix.

03 · Query path



Summary: Only the DuckDB query service touches the data, with temporary credentials for one customer. The run worker sends SQL to the Quack server. The query service swaps the user's token at STS for reader role B credentials. It reads Customer B's data and sends rows back. `customer_a/` is denied by IAM + KMS.

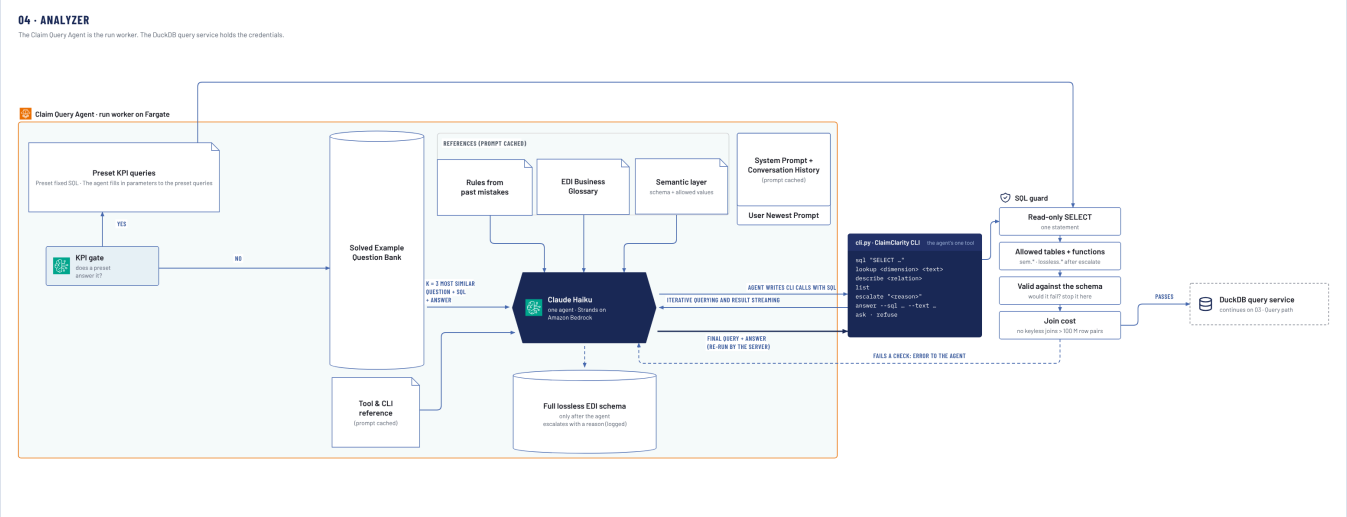
Key components:

- **Run worker (Fargate)**: holds the Claim Query Agent (Claude Haiku), `cli.py` and the read-only SQL guard.
- **Quack server (DuckDB query service, Fargate)**: receives SQL, returns rows.
- **AWS STS**: exchanges the token for temp credentials for reader role B (gate 5: trust policy allows this token).
- `semantic_layer/` : gold layer views, read-only (gate 6: IAM + KMS for reader role B).
- `lossless.*` : silver layer parquet, read only after escalation (gate 6).
- `customer_a/` : denied by IAM + KMS.

Notes / TODO:

- The old page said the STS credentials last 15 minutes. No board shows that number. Confirm it against the real role config.
- Verify the reader role trust policy actually checks the token's customer.

04 · Analyzer



Summary: A user question first hits the KPI gate. If a preset query answers it, that runs. If not, the Claim Query Agent writes CLI calls with SQL, which must pass the SQL guard before going to the DuckDB query service. The agent queries step by step, and the server re-runs the final query and answer.

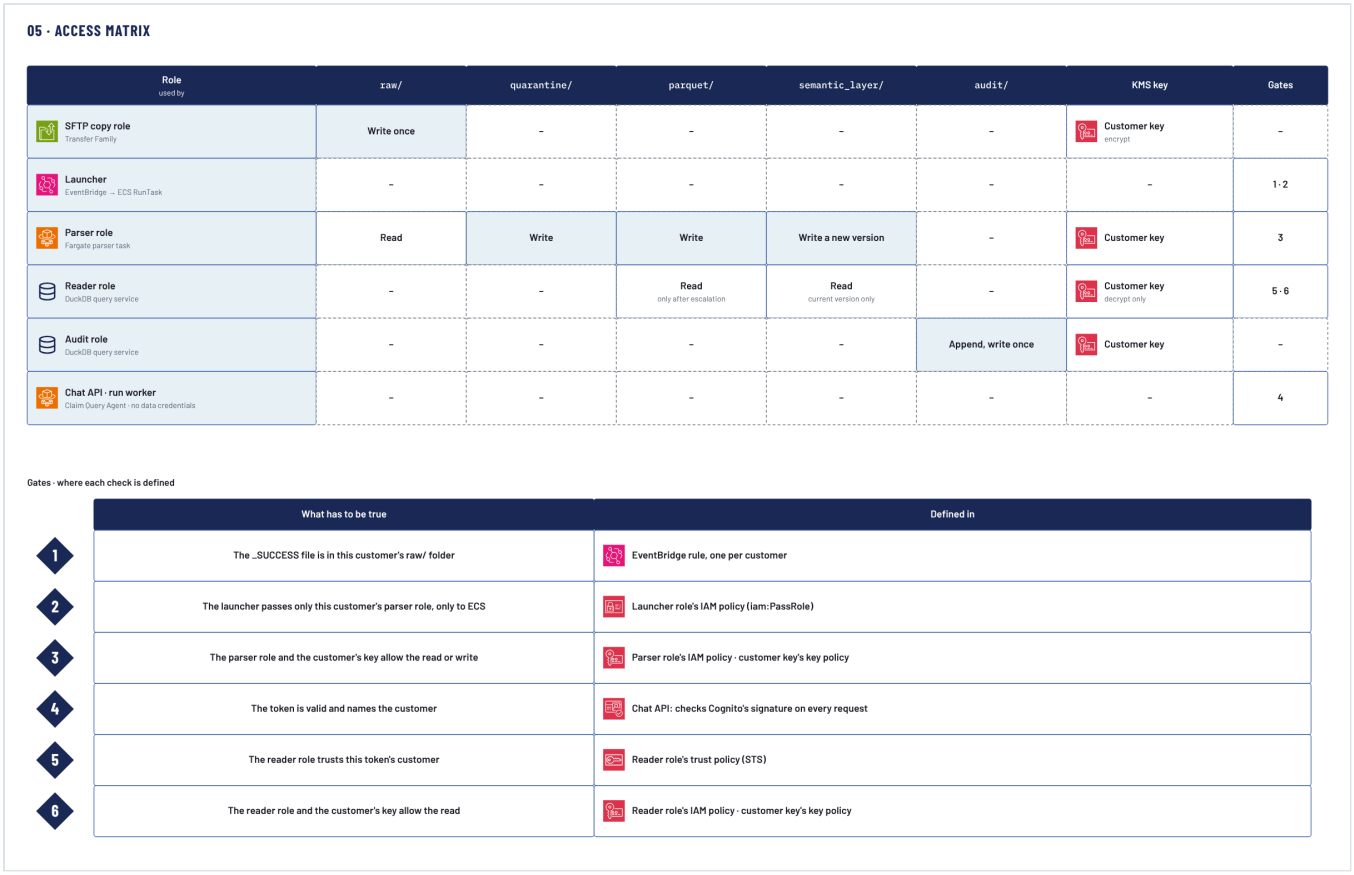
Key components:

- **KPI gate**: "does a preset answer it?" Yes runs a preset KPI query. No hands the question to the agent.
- **Claim Query Agent**: one agent, Claude Haiku, Strands on Amazon Bedrock. Runs as the run worker on Fargate. Gets the system prompt and conversation history (prompt cached) plus the newest user prompt.
- **References (prompt cached)**: preset KPI queries, solved examples / question bank (K = 3 most similar question + SQL + answer), tool & CLI reference, rules from past mistakes, EDI business glossary, semantic layer schema + allowed values.
- **Full lossless EDI schema**: given only after the agent escalates with a reason, which is logged.
- `cli.py` **(ClaimClarity CLI)**: the agent's one tool. Commands: `sql "SELECT …"`, `lookup <dimension> <text>`, `describe <relation>`, `list`, `escalate "<reason>"`, `answer --sql … --text …`, `ask`, `refuse`.
- **SQL guard**: checks read-only SELECT (one statement), allowed tables and functions (`sem.*`, `lossless.*` after escalate), valid against the schema, and join cost (no keyless joins over 100 M row pairs). A failed check sends an error back to the agent.
- **DuckDB query service**: holds the credentials. Continues on 03 · Query path.

Notes / TODO:

- The board doesn't say what `ask` and `refuse` do. Document them.
- Is there a limit on how many query steps the agent can take?

05 · Access matrix



05 · ACCESS MATRIX

Role (used by)	raw/	quarantine/	parquet/	semantic_layer/	audit/	KMS key	Gates
SFTP copy role (Transfer Family)	Write once	–	–	–	–	Customer key (encrypt)	–
Launcher (EventBridge → ECS RunTask)	–	–	–	–	–	–	1 · 2
Parser role (Fargate parser task)	Read	Write	Write	Write a new version	–	Customer key	3
Reader role (DuckDB query service)	–	–	Read (only after escalation)	Read (current version only)	–	Customer key (decrypt only)	5 · 6
Audit role (DuckDB query service)	–	–	–	–	Append, write once	Customer key	–
Chat API · run worker (Claim Query Agent · no data credentials)	–	–	–	–	–	–	4

Gates · where each check is defined

	What has to be true	Defined in
1	The _SUCCESS file is in this customer's raw/ folder	EventBridge rule, one per customer
2	The launcher passes only this customer's parser role, only to ECS	Launcher role's IAM policy (iam:PassRole)
3	The parser role and the customer's key allow the read or write	Parser role's IAM policy · customer key's key policy
4	The token is valid and names the customer	Chat API: checks Cognito's signature on every request
5	The reader role trusts this token's customer	Reader role's trust policy (STS)
6	The reader role and the customer's key allow the read	Reader role's IAM policy · customer key's key policy

Summary: What each role can do in each customer folder, which key it uses and which gates guard it. A second table lists the six gates and where each check is defined.

Key components:

- **SFTP copy role (Transfer Family)**: write once to `raw/`. Customer key: encrypt.
- **Launcher (EventBridge → ECS RunTask)**: no data access. Gates 1 and 2.
- **Parser role (Fargate parser task)**: read `raw/`, write `quarantine/` and `parquet/`, write a new version of `semantic_layer/`. Customer key. Gate 3.
- **Reader role (DuckDB query service)**: read `parquet/` only after escalation, read `semantic_layer/` current version only. Customer key: decrypt only. Gates 5 and 6.
- **Audit role (DuckDB query service)**: append, write once to `audit/`. Customer key.
- **Chat API · run worker (Claim Query Agent)**: no data credentials. Gate 4.

The six gates:

- **Gate 1**: the `_SUCCESS` file is in this customer's `raw/` folder. Defined in the EventBridge rule, one per customer.
- **Gate 2**: the launcher passes only this customer's parser role, only to ECS. Defined in the launcher role's IAM policy (`iam:PassRole`).
- **Gate 3**: the parser role and the customer's key allow the read or write. Defined in the parser role's IAM policy and the key policy.
- **Gate 4**: the token is valid and names the customer. The Chat API checks Cognito's signature on every request.

- **Gate 5**: the reader role trusts this token's customer. Defined in the reader role's trust policy (STS).
- **Gate 6**: the reader role and the customer's key allow the read. Defined in the reader role's IAM policy and the key policy.

Notes / TODO:

- The audit role's KMS column says only "Customer key" (no encrypt/decrypt). Confirm what it needs.
- Verify every cell against the real IAM, KMS and EventBridge config, and keep this list in sync with the PNG.